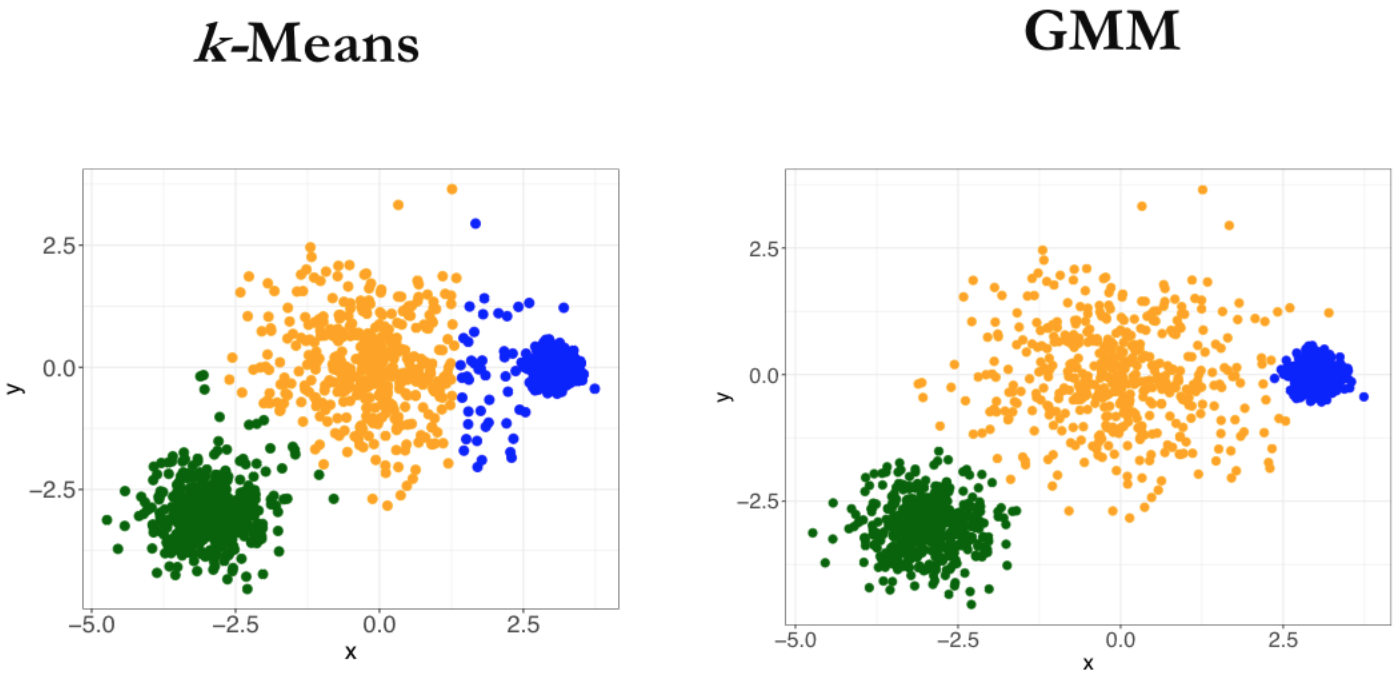
L12 EM algorithm and density estimation

Announcements

- Midterm project will be released soon. If you want to work with a team up to size three, find group mates now!

Comparing GMMs and k-Means

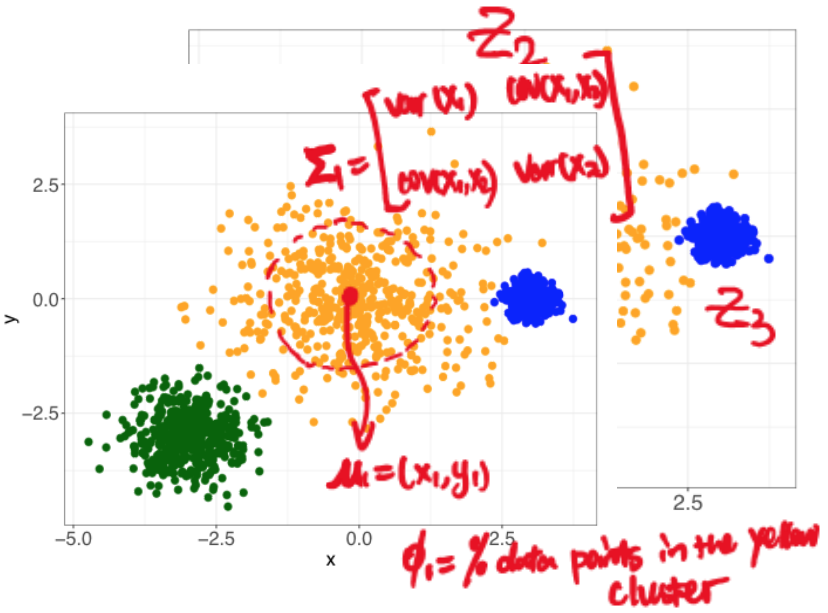


Unsupervised Gaussian Mixture Models Setup

- K distinct multivariate Gaussian distributions generating data: $P(x|z = k)$ = $N(\boldsymbol{u_k}, \boldsymbol{\Sigma_k})$
  - Unknown means, variances, and correlation structures: need to learn$(\boldsymbol{u_k}, \boldsymbol{\Sigma_k})$
- Unlabeled data -> unknown cluster mixture: $P_\theta(z = k) = \phi_k$

Gaussian Mixture Models Notation

- Let z be the index for clusters (colors)
- For each cluster z, we model the data as a multivariate Gaussian distribution
  - $P(x|z=k) = N(\boldsymbol{u_k}, \boldsymbol{\Sigma_k})$
  - $x, \boldsymbol{u}_k \in \mathbb{R}^d, \boldsymbol{\Sigma_k} \in \mathbb{R}^{d \times d}$



- Gaussian Mixture

$$P_\theta(x, z=k) = P_\theta(x|z=k) P_\theta(z=k)$$

  - where $P_\theta(z=k) = \phi_k$
  - The parameters θ are the $\mathbf{u}_k, \boldsymbol{\Sigma}_k, \phi_k$ for all k=1,2,…,K

GMM for Clustering

$z^{(i)}$ is the cluster label for data point $x^{(i)}$

- can be obtained from model parameters by calculating

$$P\left(z^{(i)} = k \middle| x^{(i)}\right) = \frac{P_\theta(Z=k,X)}{P_\theta(X)}.$$

- Bayes rule
  - $P_\theta(x, z = k) = P_\theta(x|z = k)P_\theta\,(z = k)$

$$P_\theta(x) = \sum_{k=1}^{K} P_\theta(z = k)P_\theta(x|z = k)$$
$$= \phi_1 \mathcal{N}(x; \mu_1, \Sigma_1) + \ldots + \phi_K \mathcal{N}(x; \mu_K, \Sigma_K)$$

Learn GMM from unlabeled data

Jointly learn cluster parameters and labels:

1. Objective marginalized over the cluster labels $z^{(i)}$.
2. Optimizer jointly infers both labels $z^{(i)}$ and cluster parameters μ_k, Σ_k.

$$\max_{\theta} \frac{1}{n} \sum_{i=1}^{n} \log \left(\sum_{z \in \mathcal{Z}} P_\theta(x^{(i)}, z) \right)$$

- The above objective is hard to solve, so we use the EM algorithm

Expectation Maximization (EM): Intuition

$$\max_{\theta} \frac{1}{n} \sum_{i=1}^{n} \log\left(\sum_{z \in \mathcal{Z}} P_\theta(x^{(i)}, z) \right)$$

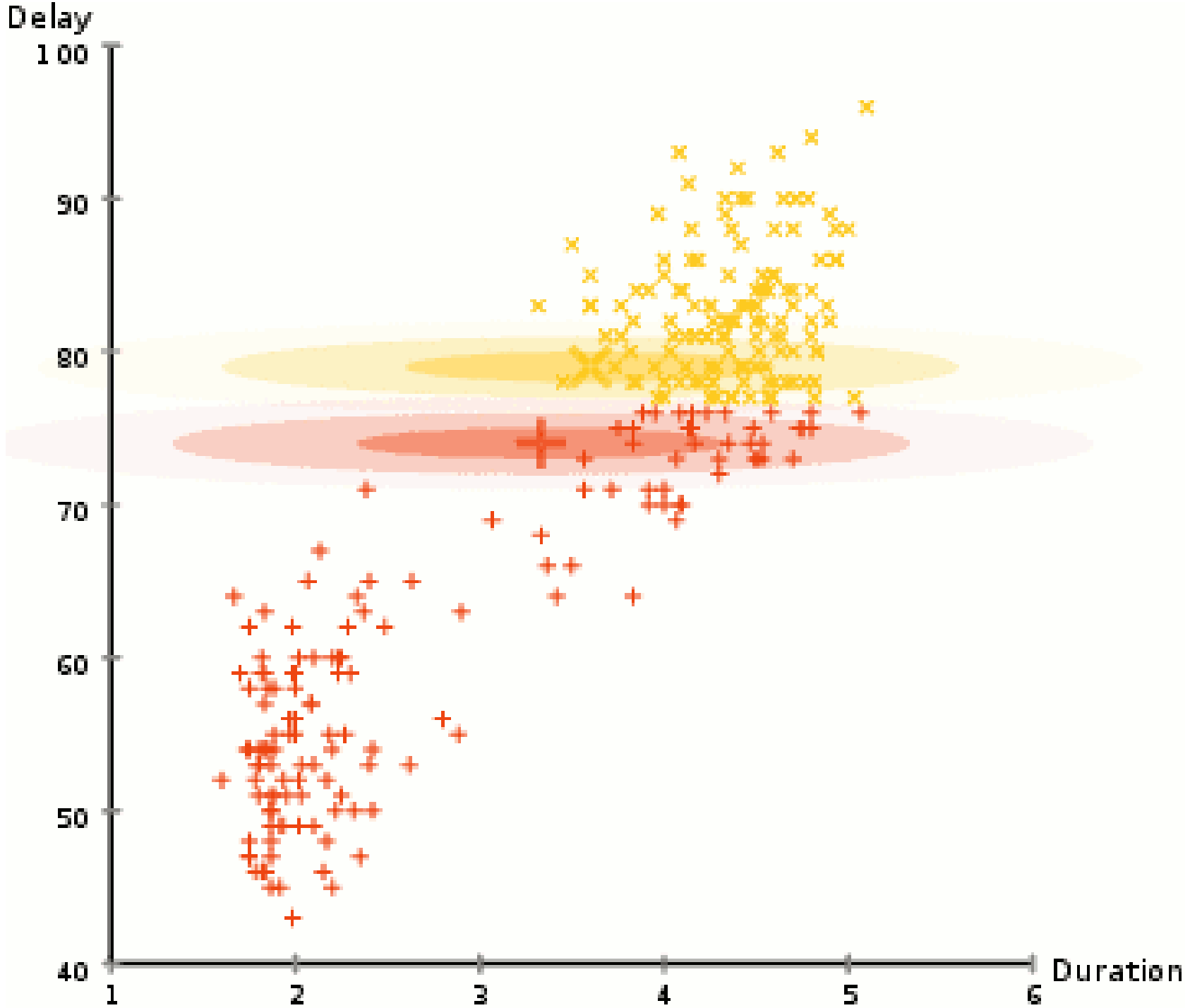
- If we know the true $z^{(i)}$ for each $x^{(i)}$, we maximize

$$\max_{\theta} \sum_{x^{(i)}, z^{(i)} \in \mathcal{D}} \log\left(P_\theta(x^{(i)}, z^{(i)}) \right).$$

- If we know θ, we can estimate the cluster assignments $z^{(i)}$ for each i by computing $P_\theta(z|x^{(i)})$.

Expectation Maximization (EM):

- Randomly initialize θ
- Repeat until convergence:
  - (**E-step**): Given the current estimate θ_t compute $P_\theta\left(z\middle|x^{(i)}\right)$. Use it to obtain cluster assignment $z^{(i)}$.
    - Similar to making a prediction GDA
  - (**M-step**): Find a new θ_{t+1} that maximizes the marginal log-likelihood by optimizing $P_\theta\left(\mathrm{z}^{(\mathrm{i})}, \mathrm{x}^{(i)}\right)$, given the $z^{(i)}$ from E-step.
    - Similar to calculating the mean and covariance for a fixed cluster assignment


Delay
100
90
80
70
60
50
40
1
2
3
4
5
6
Duration

E-Step

- Given the learned/initialized parameters $\boldsymbol{\phi}, \boldsymbol{\mu}, \boldsymbol{\Sigma}$
- Assign cluster $z^{(i)}$ in the E-step
- $P_{\theta_t}\left(z|x^{(i)}\right)$ defined "soft" assignment
  - Compute a vector of class probabilities for each $x^{(i)}$

$$p\left(z^{(i)} = j|x^{(i)}; \phi, \mu, \Sigma\right) =$$

$$\frac{p\left(x^{(i)}|z^{(i)} = j; \mu, \Sigma\right) p\left(z^{(i)} = j; \phi\right)}{\sum_{l=1}^{k} p\left(x^{(i)}|z^{(i)} = l; \mu, \Sigma\right) p\left(z^{(i)} = l; \phi\right)}$$

M-Step

- Given the cluster assignment probabilities $P_{\theta_t}\left(z|x^{(i)}\right)$, we update the model parameters $\boldsymbol{\phi}, \boldsymbol{\mu}, \boldsymbol{\Sigma}$ that maximizes the expected likelihood of the data under this assignment

$$\theta_{t+1} = \arg\max_{\theta} \sum_{i=1}^{n} \mathbb{E}_{z^{(i)} \sim P_{\theta_t}(z|x^{(i)})} \log P_{\theta}(x^{(i)}, z^{(i)})$$

- Tr

Deriving the M-step

At the M-step, we optimize the expected log-likelihood of our model.

$$\max_\theta \sum_{x \in D} \mathbb{E}_{z \sim P_{\theta_t}(z|x)} \log P_\theta(x, z) =$$

$$\max_\theta \left(\sum_{k=1}^{K} \sum_{x \in D} P_{\theta_t}(z_k|x) \log P_\theta(x|z_k) + \sum_{k=1}^{K} \sum_{x \in D} P_{\theta_t}(z_k|x) \log P_\theta(z_k) \right)$$

if data were drawn according to our current guess.

- As in supervised learning, we can optimize the two terms above separately

Find mean and covariance for each cluster

- We will start with $P_\theta(x \mid z = k) = N(x; \mu_k, \Sigma_k)$.
- Find μ_k, Σ_k that optimize

$$\max_\theta \sum_{x^{(i)} \in D} P(z = k|x^{(i)}) \log P_\theta(x^{(i)}|z = k)$$

- This corresponds to fitting a Gaussian to a dataset whose elements $x^{(i)}$ each have a weight $P(z = k|x^{(i)})$

Find mean and covariance for each cluster

- Compute the derivative, set it to zero, and obtain closed form solutions

$$\mu_k = \frac{\sum_{i=1}^{n} P(z = k|x^{(i)})x^{(i)}}{n_k}$$

$$\Sigma_k = \frac{\sum_{i=1}^{n} P(z = k|x^{(i)})(x^{(i)} - \mu_k)(x^{(i)} - \mu_k)^\top}{n_k}$$

$$n_k = \sum_{i=1}^{n} P(z = k|x^{(i)})$$

- Empirical mean and covariance of the dataset D when each element $x^{(i)}$ has a weight $P(z = k|x^{(i)})$.

Find mean and covariance for each cluster

- Similarly, the class priors are

$$\phi_k = \frac{n_k}{n}$$

$$n_k = \sum_{i=1}^{n} P(z = k|x^{(i)})$$

Are the following statements on unsupervised GMM/EM algorithm correct?

0

In unsupervised GMM, it is easy to optimize the clustering assignment and the parameters of the models at the same time because the objective function of GMM is convex	0
EM algorithm is an iterative procedure that guarantees to solve the log-likelihood objective to optimality	0
EM algorithm guarantees the same solution regardless of the cluster initialization	0
We compute $P(Z\|X^{(i)})$ at the E-step so we can connect our current belief on cluster assignment with the model parameters	0
Like in K-means, EM in GMM assigns the cluster assignment in one step and updates the model parameter in another	0
Underfitting and overfitting in GMM can be detected through holding out datapoints	0

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

Expectation Maximization: Definition

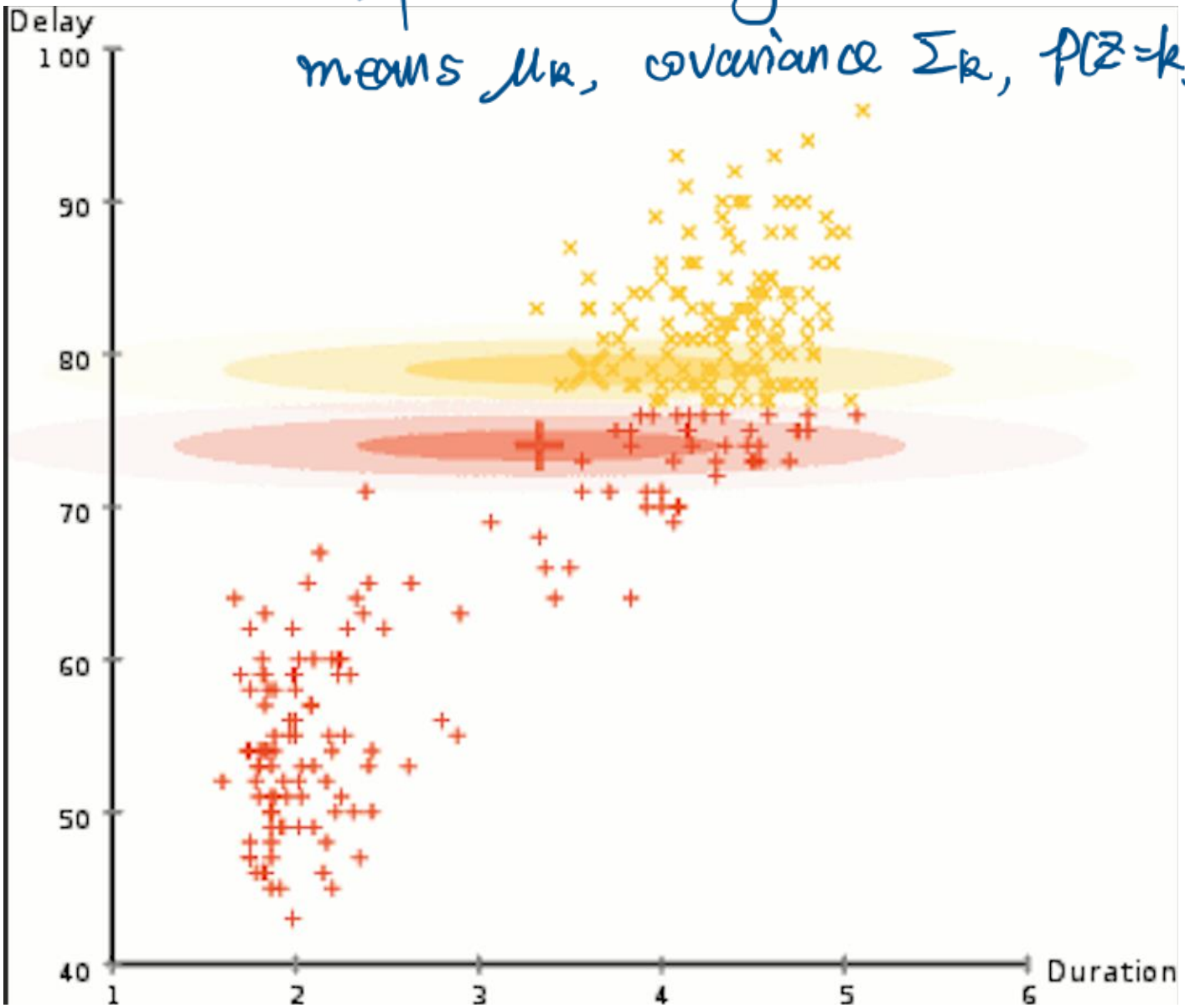
- Formally, EM learns the parameters θ of a latent-variable model $P_\theta(x, z)$ over a dataset $D = \{x^{(i)} \mid i = 1,2, \ldots, n\}$ as follows
- For t=0,1,2,… repeat until convergence:
  - (**E-Step**) For each $x^{(i)} \in D$ compute $P_{\theta_t}\left(z \middle| x^{(i)}\right)$
  - (**M-Step**) Compute new weights θ_{t+1} as

$$\theta_{t+1} = \arg\max_{\theta} \sum_{i=1}^{n} \mathbb{E}_{z^{(i)} \sim P_{\theta_t}(z|x^{(i)})} \log P_\theta(x^{(i)}, z^{(i)})$$

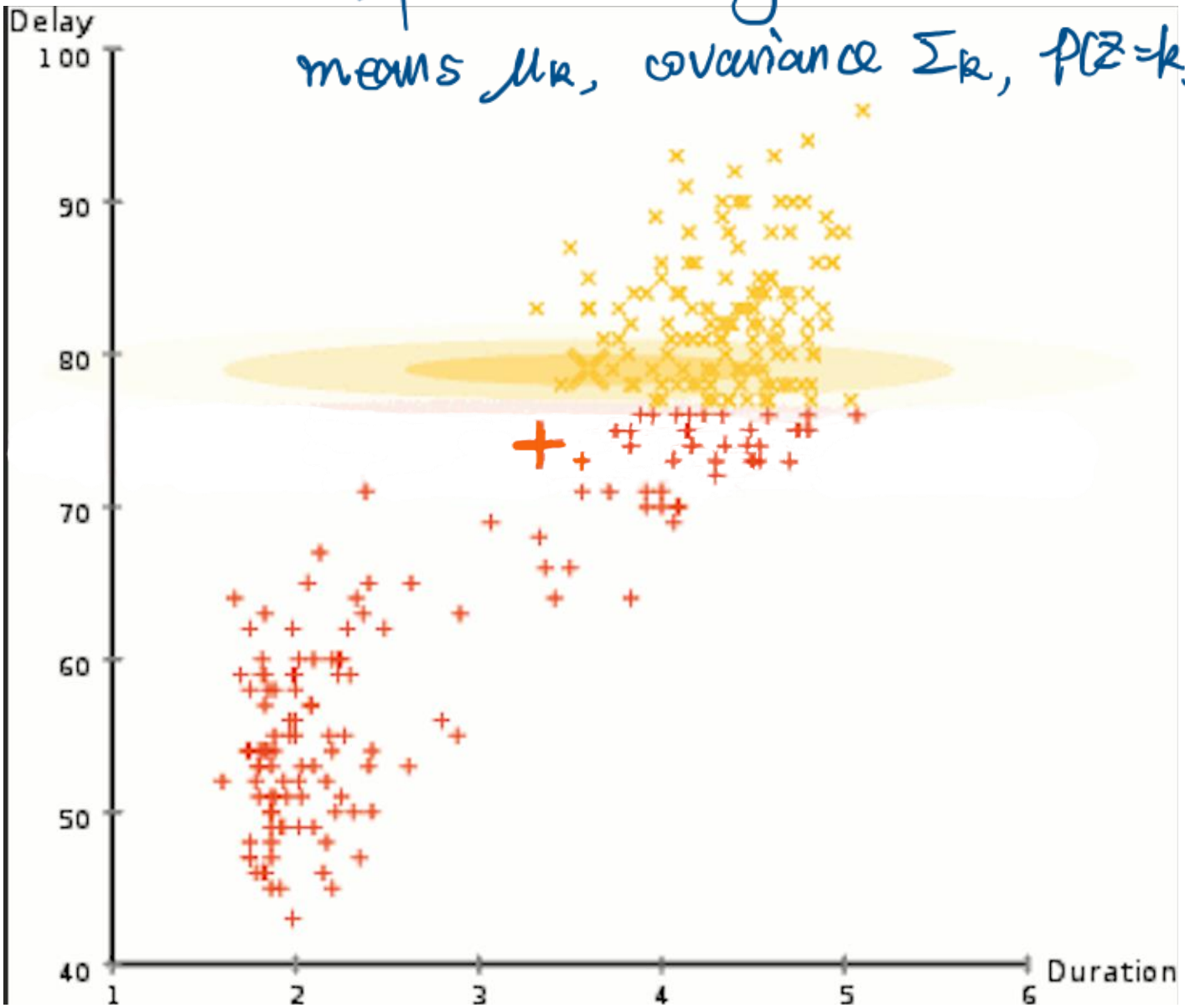
  using the previous formulas

Each assignment $P_\theta\left(z \middle| x^{(i)}\right)$ are "soft"-> M-step involves an expectation

Step 1: randomly initialize cluster means μ_k, covariance Σ_k, $P(Z=k) = \phi_k$.

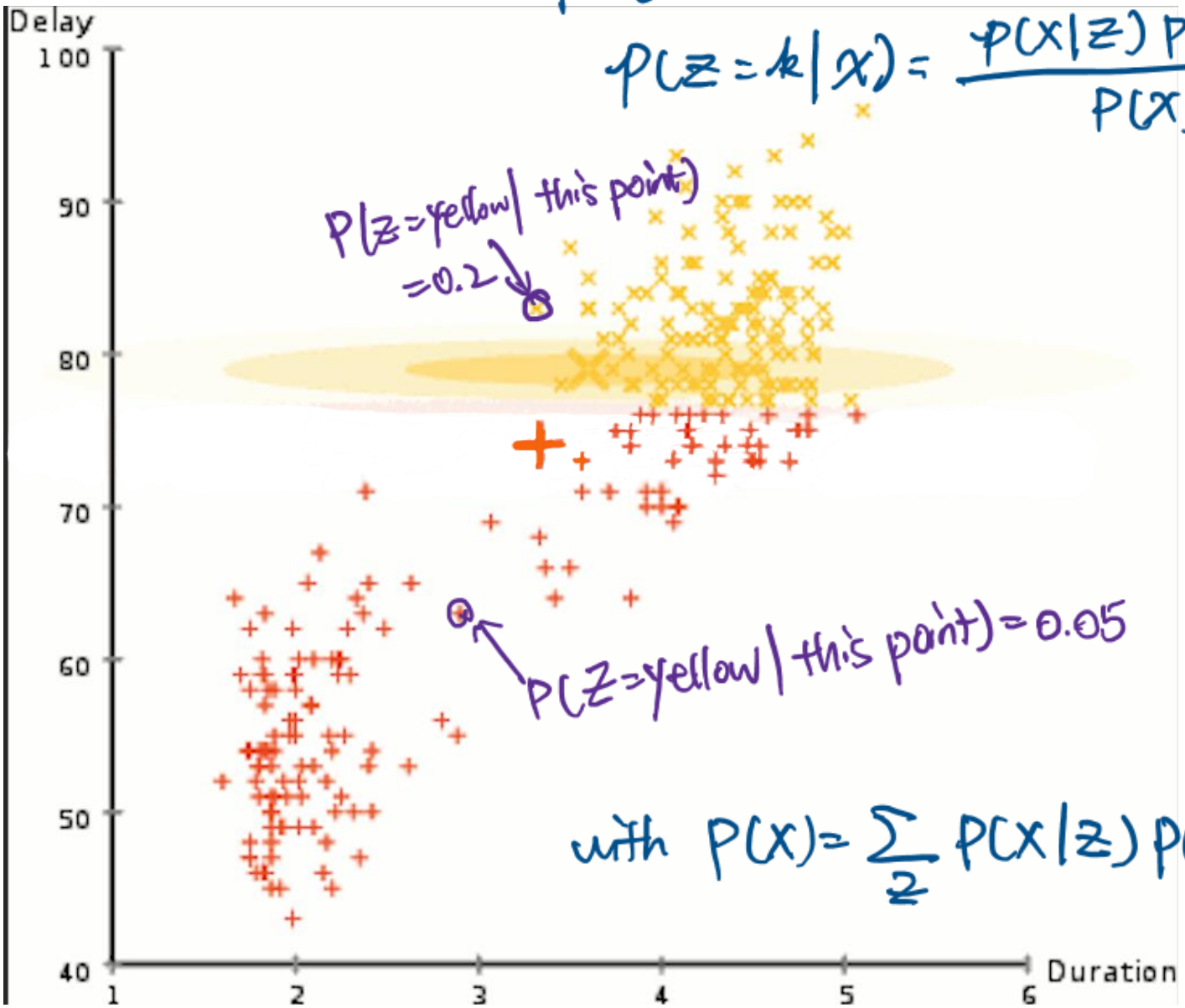


Step 1: randomly initialize cluster means μ_k, covariance Σ_k, $P(Z=k) = \phi_k$.



E-step: given μ_k, Σ_k, ϕ_k, calculate:

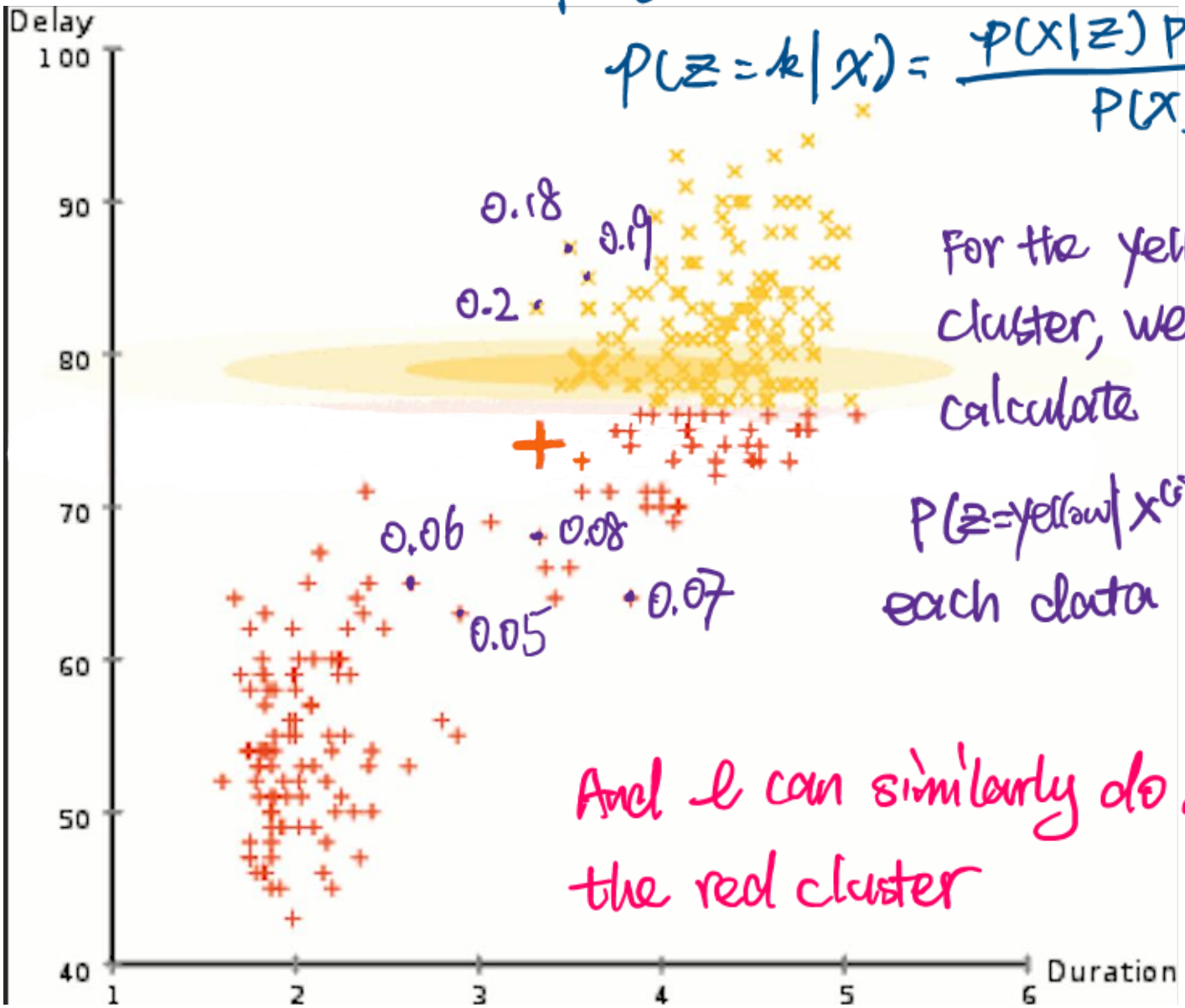
$$p(z=k|x) = \frac{p(x|z)\,p(z)}{p(x)}$$



with $p(x) = \sum_z p(x|z)\,p(z)$

E-step: given μ_k, Σ_k, ϕ_k, calculate:

$$p(z=k|x) = \frac{p(x|z)\,p(z)}{p(x)}$$



For the yellow cluster, we can calculate

$p(z=\text{yellow}|x^{(i)})$ for each data point

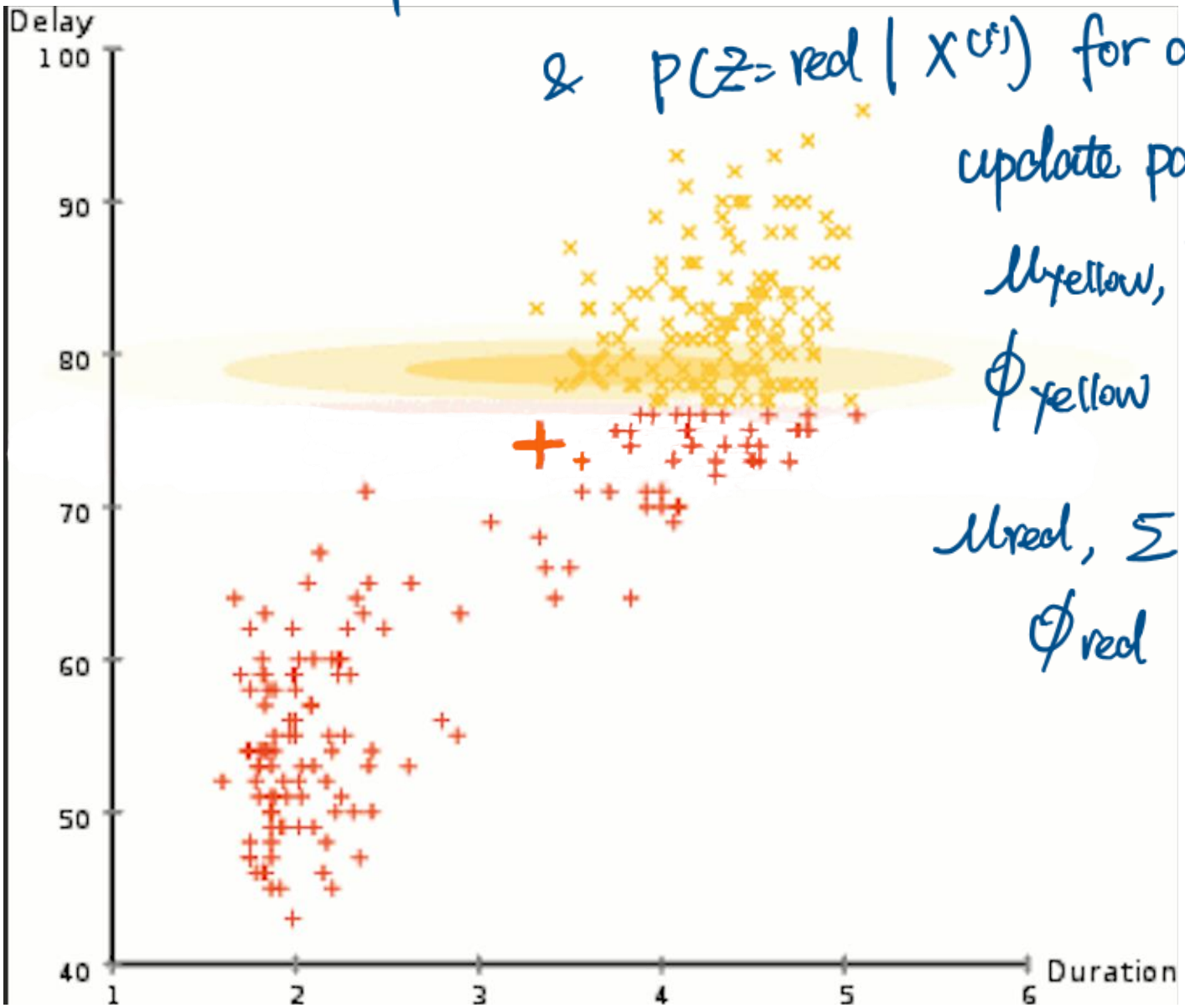
And I can similarly do so for the red cluster

M step: Given $P(z = \text{yellow} \mid x^{(i)})$ for all $x^{(i)}$ & $P(z = \text{red} \mid x^{(i)})$ for all $x^{(i)}$.

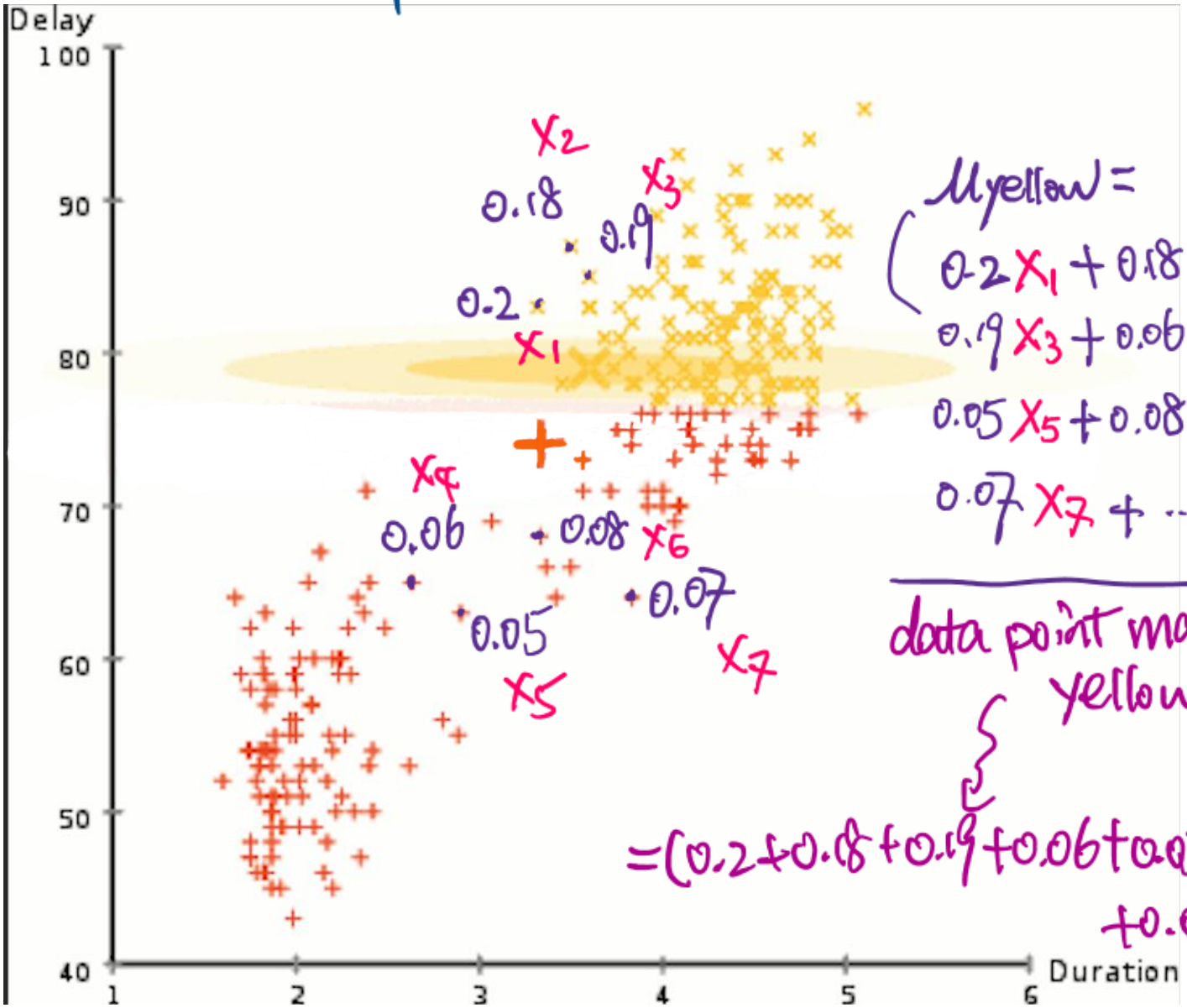
update parameters

$\mu_{\text{yellow}}, \Sigma_{\text{yellow}}, \phi_{\text{yellow}}$

$\mu_{\text{red}}, \Sigma_{\text{red}}, \phi_{\text{red}}$



M step: Given $P(z = \text{yellow} \mid x^{(i)})$ for all $x^{(i)}$



$\mu_{\text{yellow}} = ($
$0.2 x_1 + 0.18 x_2 +$
$0.19 x_3 + 0.06 x_4 +$
$0.05 x_5 + 0.08 x_6 +$
$0.07 x_7 + \cdots)$

data point mass in yellow

$= (0.2 + 0.18 + 0.19 + 0.06 + 0.05 + 0.08 + 0.07 + \cdots)$

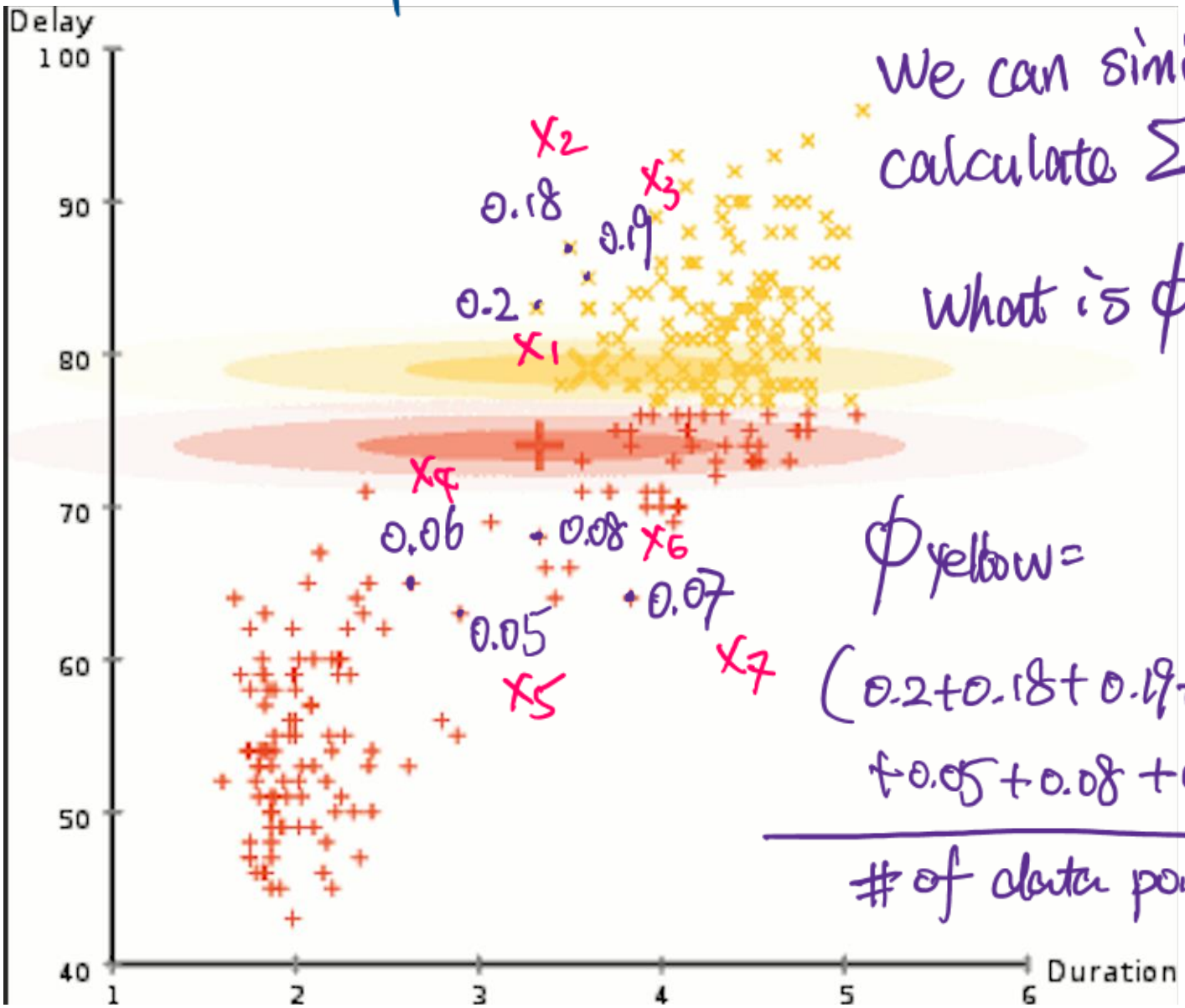
Find mean and covariance for each cluster

$$\mu_k = \frac{\sum_{i=1}^{n} P(z = k|x^{(i)})x^{(i)}}{n_k}$$

$$n_k = \sum_{i=1}^{n} P(z = k|x^{(i)})$$

Empirical mean of the dataset D when each element $x^{(i)}$ has a weight $P(z = k|x^{(i)})$.

M step: Given $P(z=\text{yellow} \mid x^{(i)})$ for all $x^{(i)}$



We can similarly calculate Σ_{yellow}.

What is ϕ_{yellow}?

$$\phi_{\text{yellow}} = \frac{(0.2+0.18+0.19+0.06+0.05+0.08+0.07+\cdots)}{\#\text{ of data points.}}$$

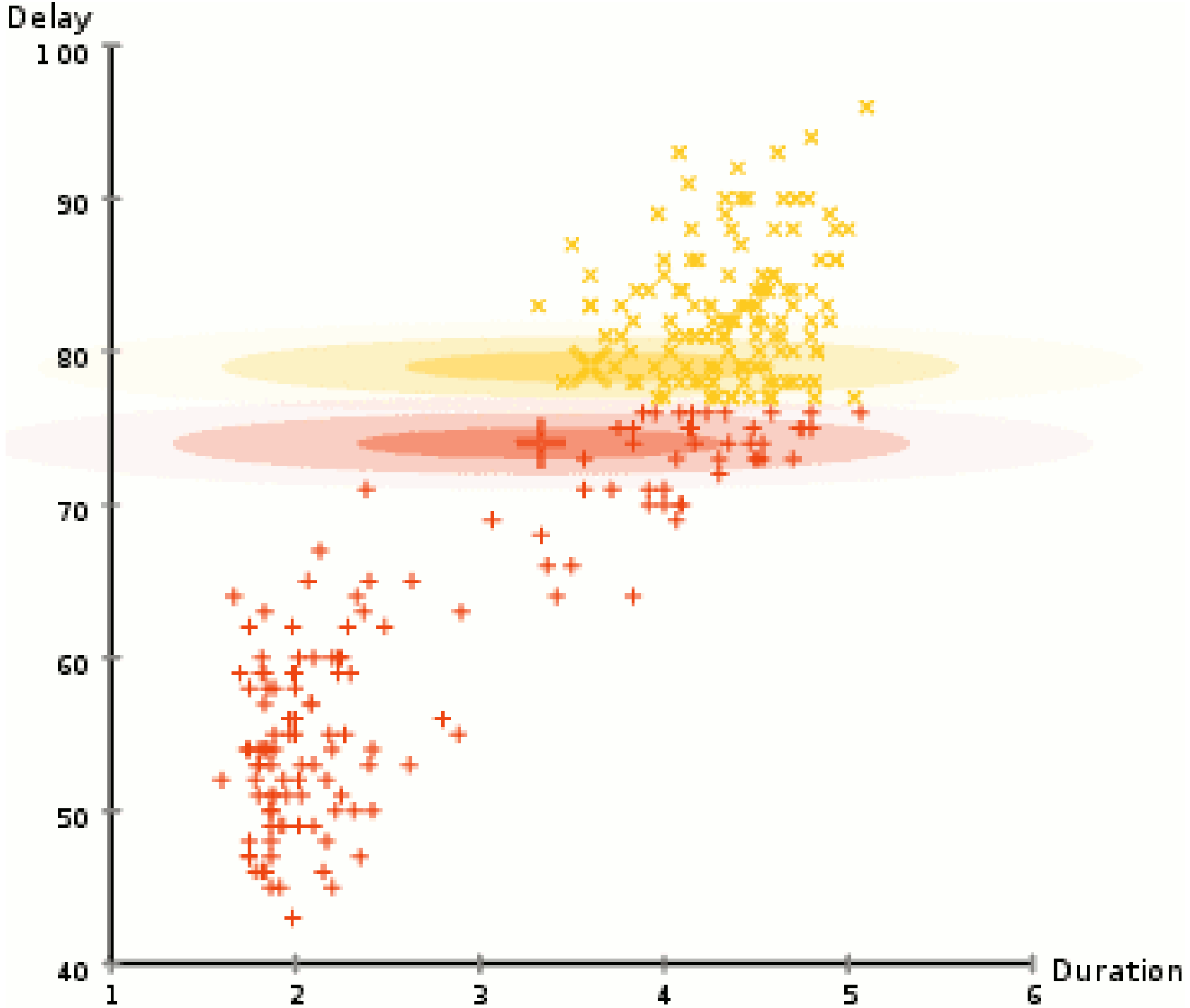
Find mean and covariance for each cluster

$$\Sigma_k = \frac{\sum_{i=1}^{n} P(z = k|x^{(i)})(x^{(i)} - \mu_k)(x^{(i)} - \mu_k)^\top}{n_k}$$

$$n_k = \sum_{i=1}^{n} P(z = k|x^{(i)})$$

- Empirical covariance of the dataset D when each element $x^{(i)}$ has a weight $P(z = k|x^{(i)})$.

$$\phi_k = \frac{n_k}{n}$$


Delay
100
90
80
70
60
50
40
1
2
3
4
5
6
Duration

Expectation Maximization: Definition

- Formally, EM learns the parameters θ of a latent-variable model $P_\theta(x, z)$ over a dataset $D = \{x^{(i)} \mid i = 1,2, \ldots, n\}$ as follows
- For t=0,1,2,… repeat until convergence:
  - (**E-Step**) For each $x^{(i)} \in D$ compute $P_{\theta_t}\left(z \middle| x^{(i)}\right)$
  - (**M-Step**) Compute new weights θ_{t+1} as

$$\theta_{t+1} = \arg\max_{\theta} \sum_{i=1}^{n} \mathbb{E}_{z^{(i)} \sim P_{\theta_t}(z|x^{(i)})} \log P_\theta(x^{(i)}, z^{(i)})$$

  using the formulas

Each assignment $P_\theta\left(z \middle| x^{(i)}\right)$ are "soft"-> M-step involves an expectation

Pros and Cons of EM

EM is a very important optimization algorithm in machine learning.

Pros:

- easy to implement and is guaranteed to converge.
- works in a lot of important ML models.

Cons:

- can get stuck in local optima.
- may not be able to compute $P_{\theta_t}\left(z\middle|x^{(i)}\right)$ in every model.
- Considerations: How to choose initial cluster centers?

Generalization in Probabilistic Models

- Consider the following synthetic dataset generated from 4 Gaussian distributions

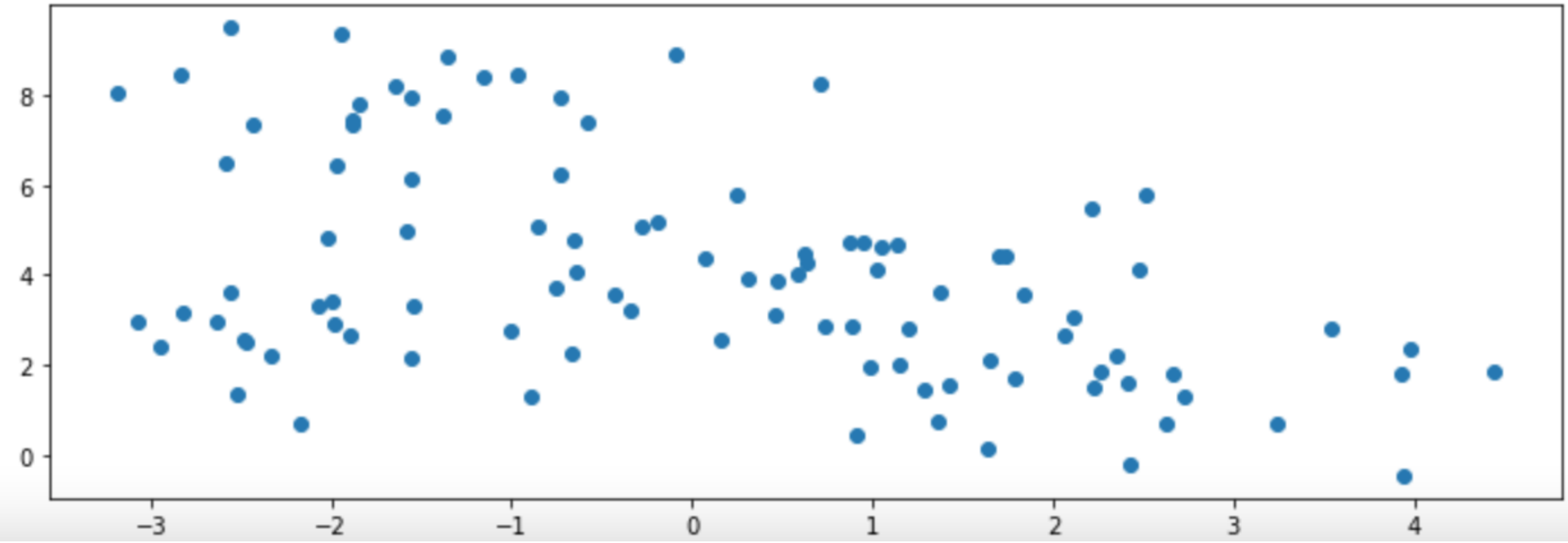
```
import numpy as np
from sklearn import datasets
from matplotlib import pyplot as plt
plt.rcParams['figure.figsize'] = [12, 4]

# generate 150 random points
np.random.seed(0)
X_all, y_all = datasets.make_blobs(150, centers=4)

# use the first 100 points as the main dataset
X, y = X_all[:100], y_all[:100]
plt.scatter(X[:,0], X[:,1])
```

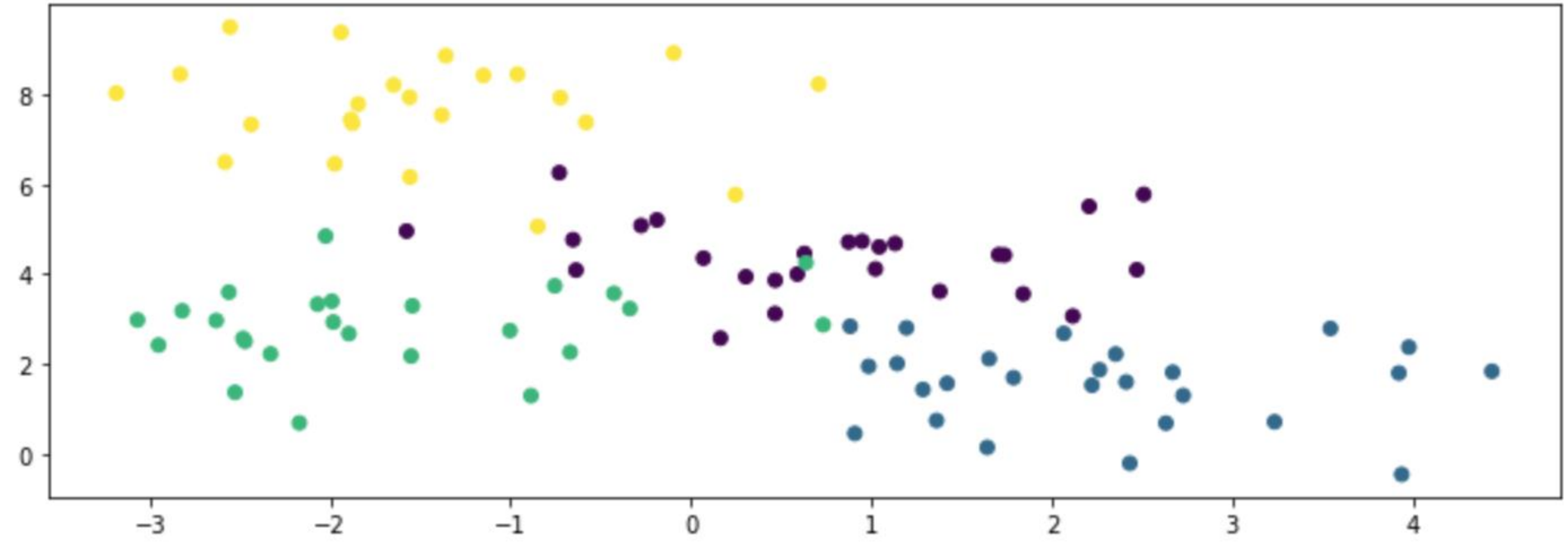
Generalization in Probabilistic Models

- Consider the following synthetic dataset generated from 4 Gaussian distributions



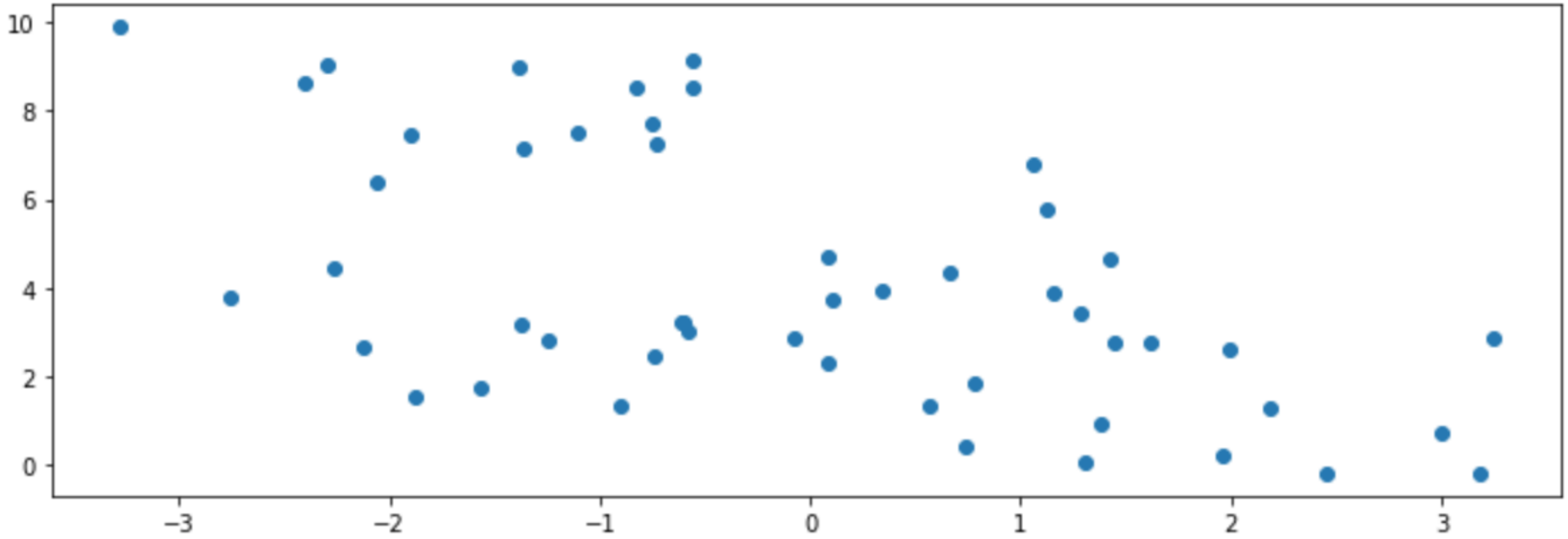
Generalization in Probabilistic Models

- Consider the following synthetic dataset generated from 4 Gaussian distributions



Generalization in Probabilistic Models

- Generate 50 holdout data points



Underfitting

- GMM: not capturing all clusters
- Can calculate objective on the training and testing data

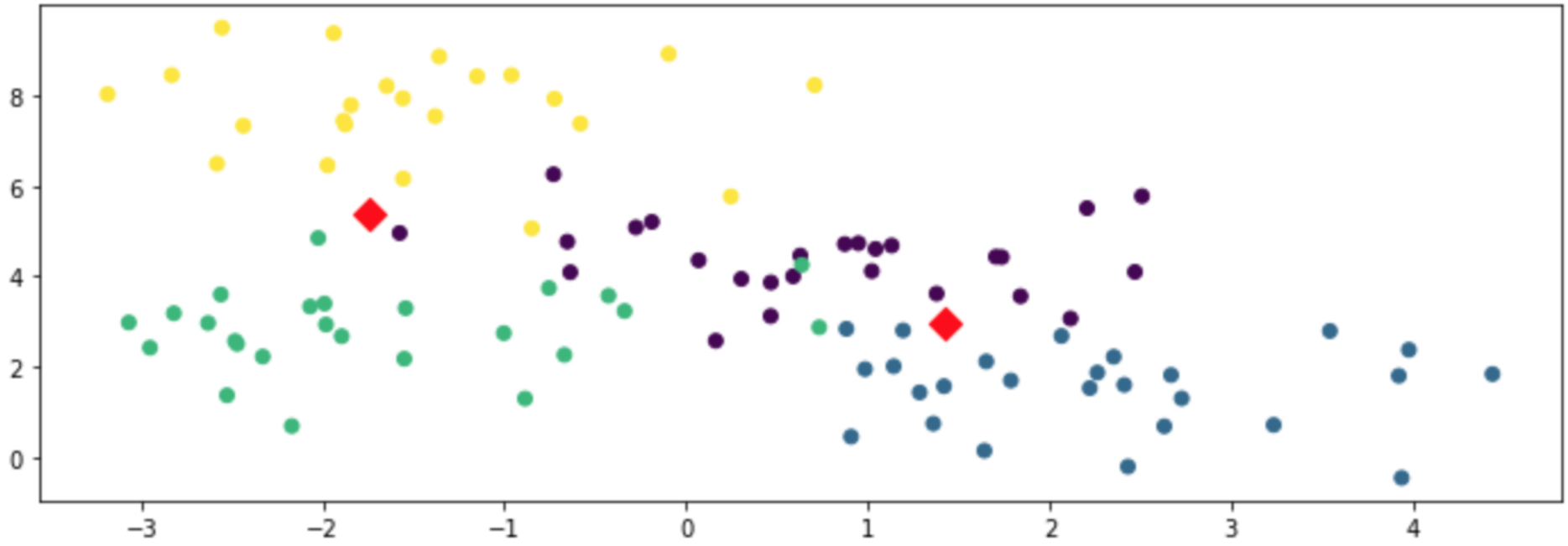
```
plt.scatter(X[:,0], X[:,1], c=y)
plt.scatter(model.means_[:,0], model.means_[:,1],
            marker='D', c='r', s=100)
print('Training Set Log-Likelihood (higher is better): %.2f'
      % model.score(X))
print('Holdout Set Log-Likelihood (higher is better): %.2f'
      % model.score(X_holdout))
```

```
Training Set Log-Likelihood (higher is better): -4.09
Holdout Set Log-Likelihood (higher is better): -4.22
```

Underfitting

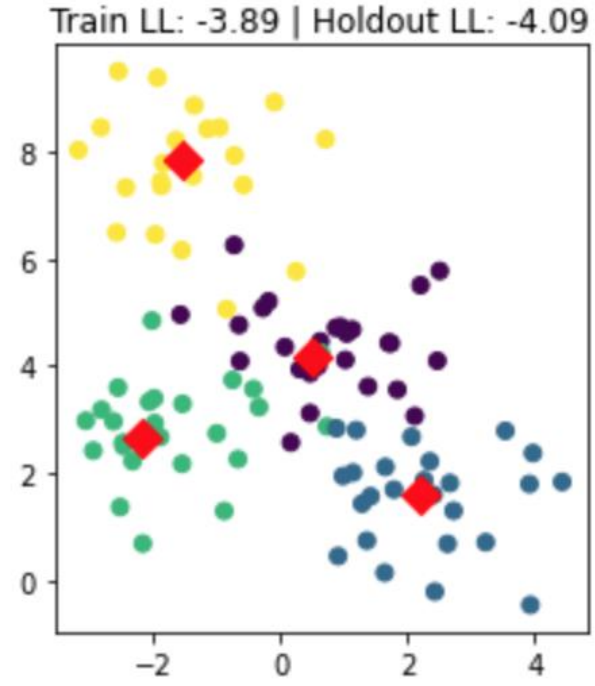
- GMM: not capturing all clusters
- Can calculate objective on the training and testing data

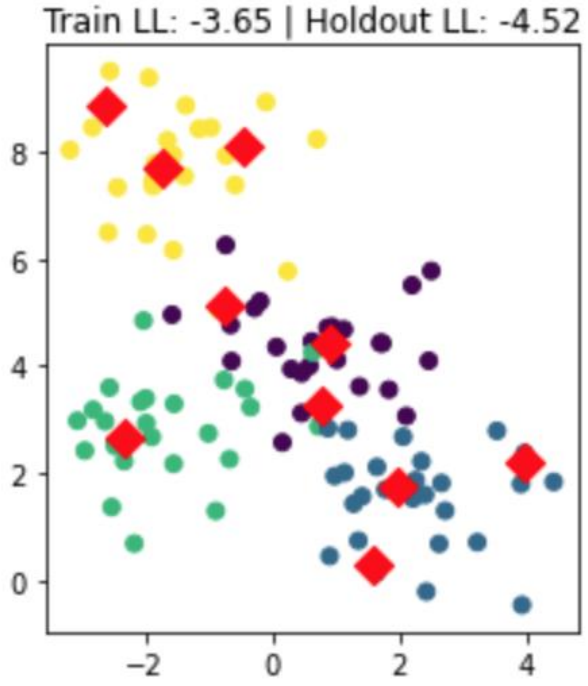
```
Training Set Log-Likelihood (higher is better): -4.09
Holdout Set Log-Likelihood (higher is better): -4.22
```

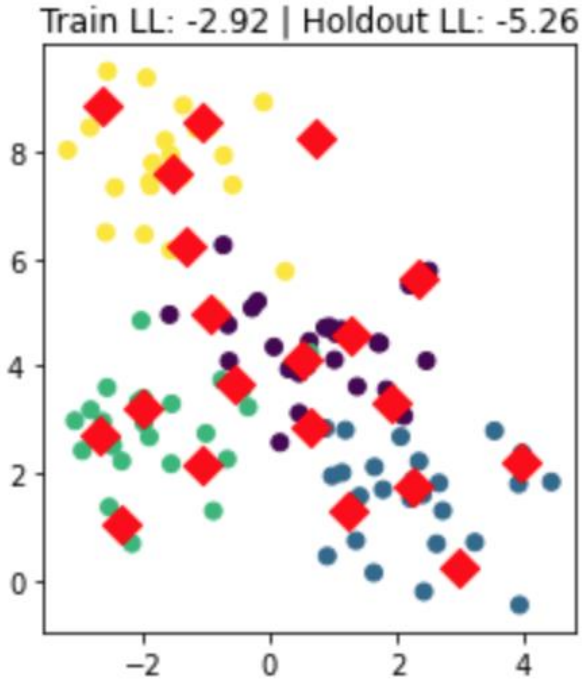


Overfitting

- Increase the number of clusters

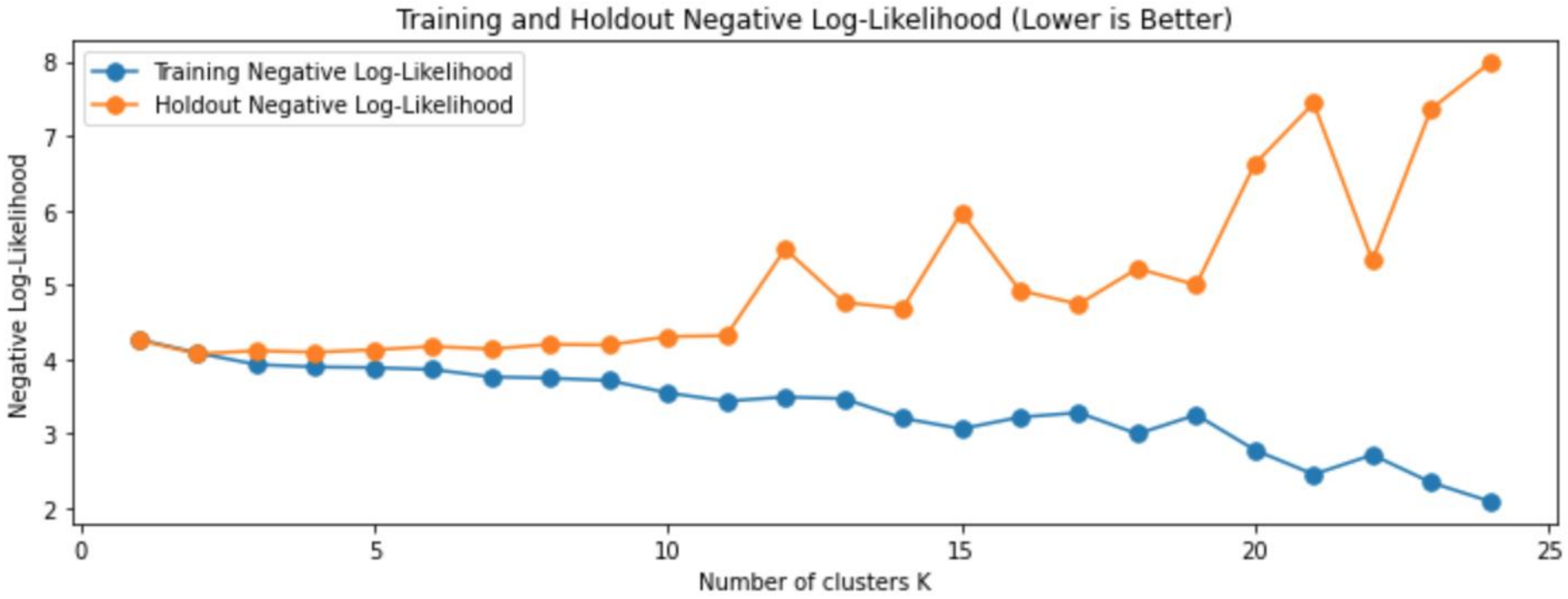






Measuring Generalization Using Log-Likelihood

- Probabilistic unsupervised models optimize an objective that can be used to detect overfitting and underfitting by comparing performance between training and holdout sets.

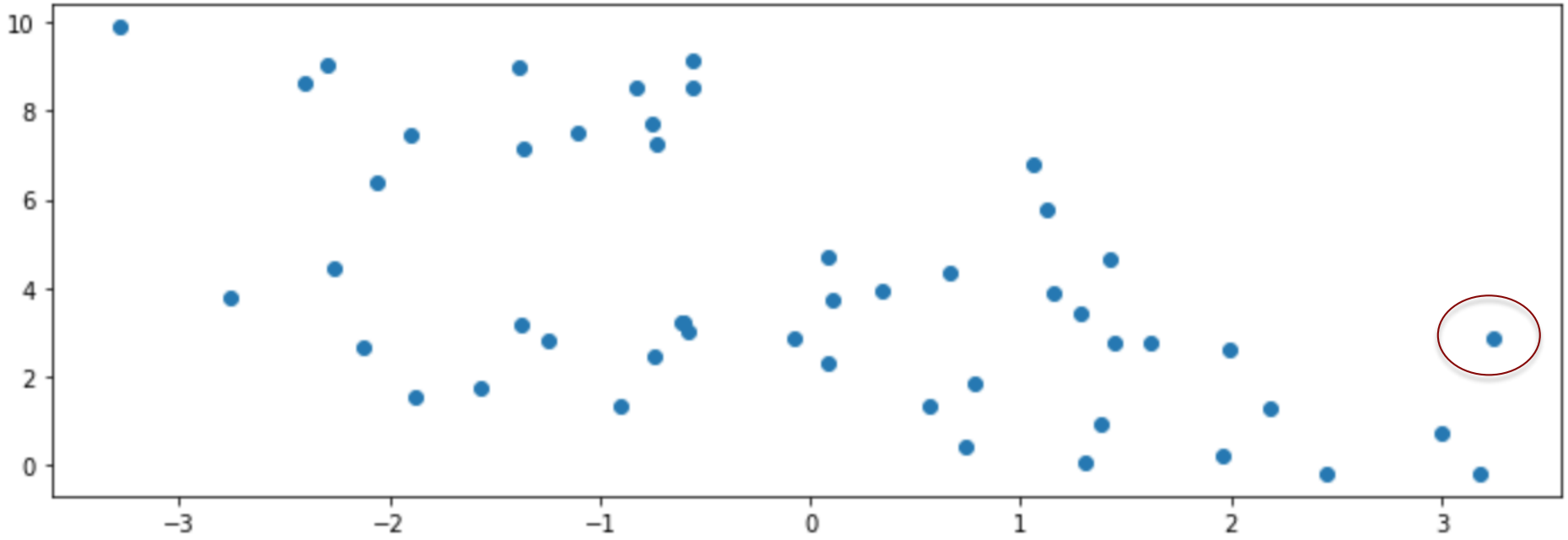


Density Estimation

- Given a set of data $D = \{x^{(i)} \mid i = 1,2, \ldots, n\}$
  - Estimate the probability distribution of the data
  - E.g., Gaussian -> learning the mean and covariance
- GMM: density estimation + clustering

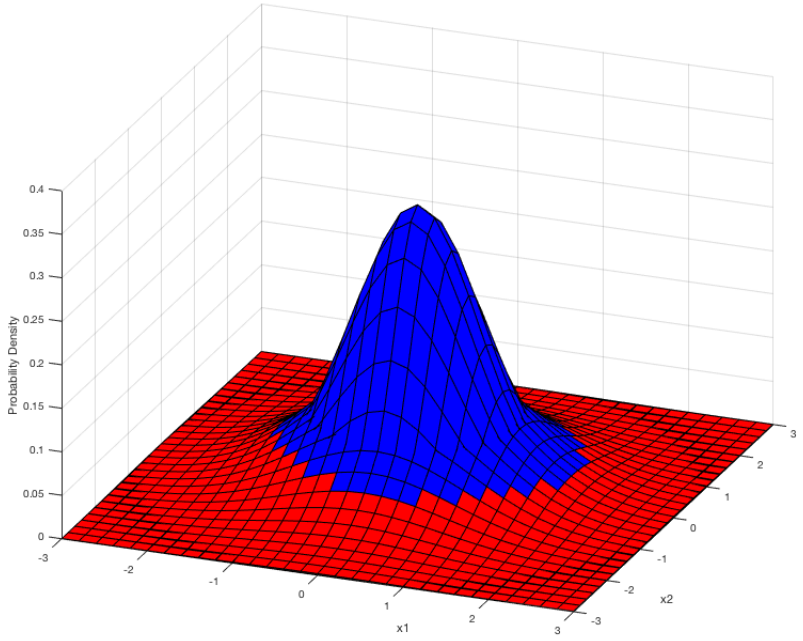
Application: Outlier Detection

- Is the red point an outlier or not?
- Assume data is IID draw from distribution P_{data}



Density Estimation

- Density estimation: approximating P_{data} with P_θ
- Outlier detection: given x', calculate $P_\theta(x')$ to decide whether x' is an outlier



How to find P_θ?

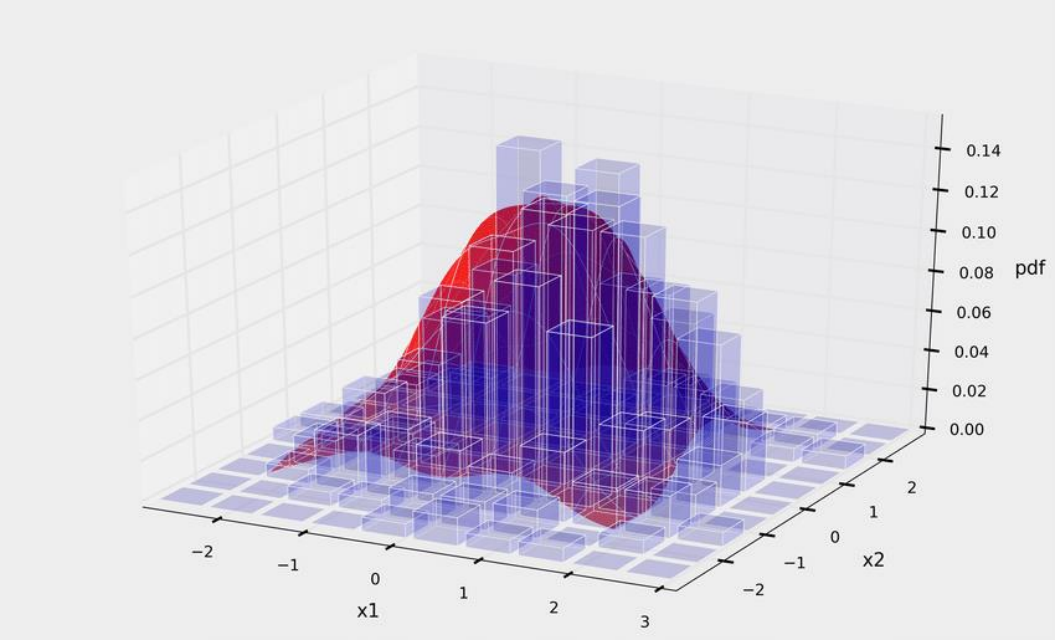
- Maximum Likelihood Estimation
  - Model parametrized by θ
  - Log-likelihood objective:

$$\max_\theta \ell(\theta) = \max_\theta \frac{1}{n} \sum_{i=1}^{n} \log P_\theta(x^{(i)}).$$

  - choose θ that maximizes the probability of seeing my data

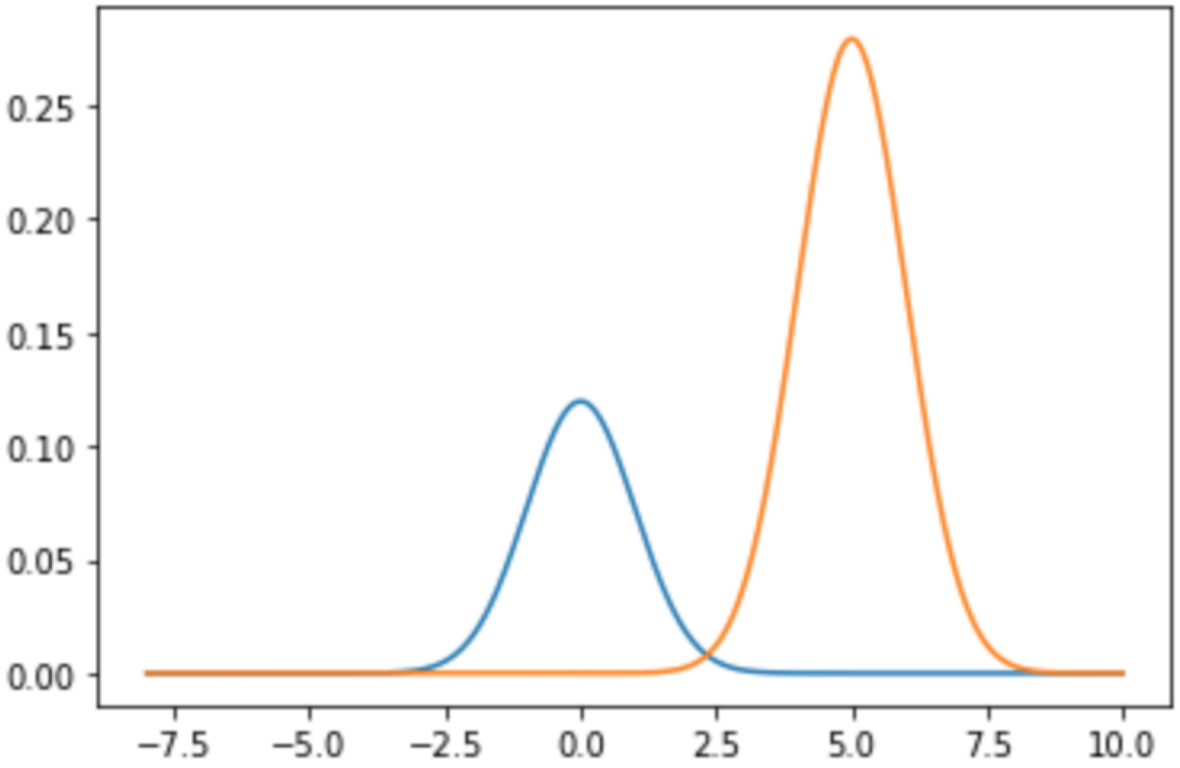
What if we do not want to parametrize the model?

- Can we learn the density of P_{data} from data alone?
  - Yes! We can take a non-parametric approach
- Example:
  - Histogram density estimation



Histogram Density Estimation

- Example: 20 data points generating from mixture of two 1D Gaussians:
  - $P_{data}(x) = 0.3 * N(x; \mu = 0, \sigma = 1) + 0.7 * N(x; \mu = 5, \sigma = 1)$



Histogram Density Estimation

- [1.62434536 -0.61175641 -0.52817175 -1.07296862 0.86540763 -2.3015387 6.74481176 4.2387931 5.3190391 4.75062962 6.46210794 2.93985929 4.6775828 4.61594565 6.13376944 3.90010873 4.82757179 4.12214158 5.04221375 5.58281521]
- Partition the x-axis into intervals

```
print(np.linspace(-5, 10, 10))
```

```
[-5.         -3.33333333 -1.66666667  0.          1.66666667  3.33333333
  5.          6.66666667  8.33333333 10.        ]
```

- Count the number of data points in each interval

Histogram Density Estimation

- [1.62434536 -0.61175641 -0.52817175 -1.07296862 0.86540763 -2.3015387 6.74481176 4.2387931 5.3190391 4.75062962 6.46210794 2.93985929 4.6775828 4.61594565 6.13376944 3.90010873 4.82757179 4.12214158 5.04221375 5.58281521]
- Partition the x-axis into intervals

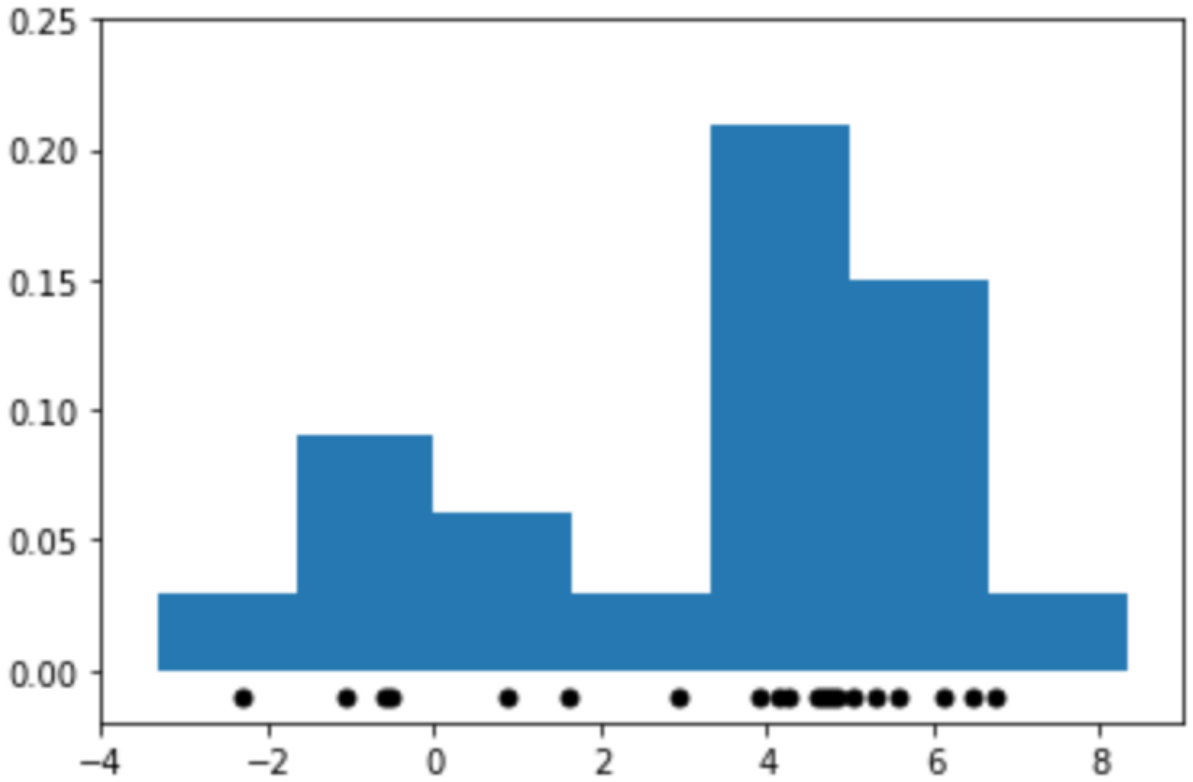
```
print(np.linspace(-5, 10, 10))
```

```
[-5.         -3.33333333 -1.66666667  0.          1.66666667  3.33333333
  5.          6.66666667  8.33333333 10.        ]
```

- Count the number of data points in each interval
- Interval width: 5/3
- Each data point has 1/20 mass
  - Height of the bin (1/20) / (5/3)

Histogram Density Estimation

```
# We set the number of bins to be equal to 10 - 1 = 9 (np.linspace gets us boundaries of the bins)
# Though the figure seems like it has less than 9 bins because some bins do not have any data
bins = np.linspace(-5, 10, 10)
plt.hist(X[:, 0], bins=bins, density=True) # plot the histogram
plt.plot(X[:, 0], np.full(X.shape[0], -0.01), '.k', markersize=10) # plot the points in X
plt.xlim(-4, 9)
plt.ylim(-0.02, 0.25)
```



Are the following statements on nonparametric estimation correct?

0

Nonparametric density estimation can be combined with generative models to learn the distribution of the data. Compared with parametric models, e.g., Gaussian distributions, nonparametric methods are more robust and flexible.

0

Nonparametric methods often require more data to estimate a target distribution well when compared with parametric methods

0

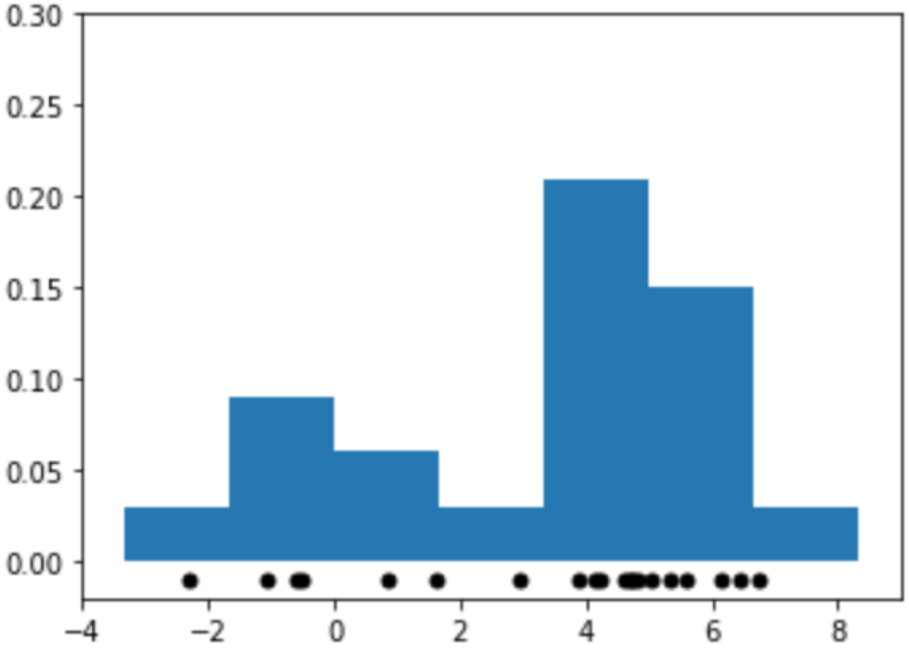
Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

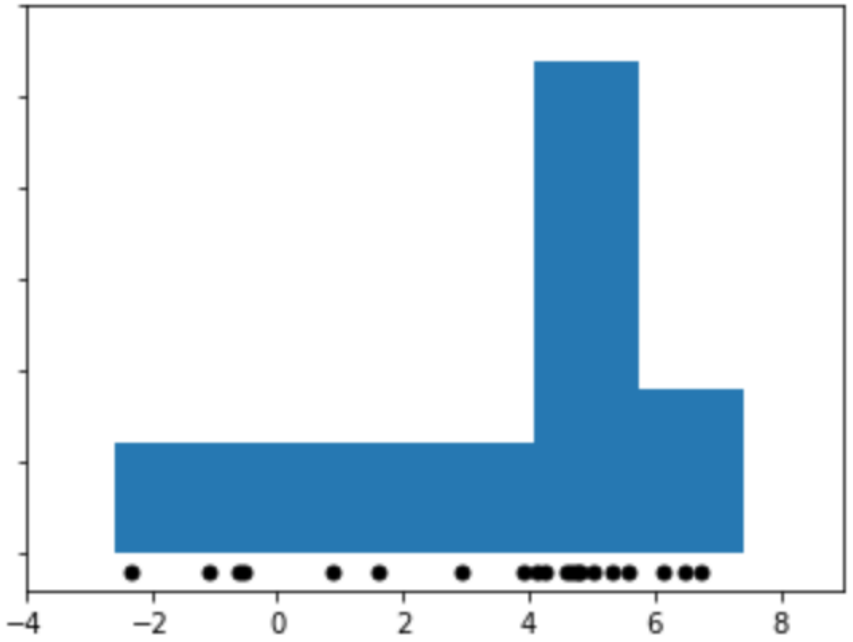
Limitations of Histograms

- The histogram is not "smooth".
  - Expect elements in the same local region of the data distribution to have somewhat similar probabilities (no sudden changes).
- The shape of the histogram depends on the bin positions and bin width.

Example: shifted bins

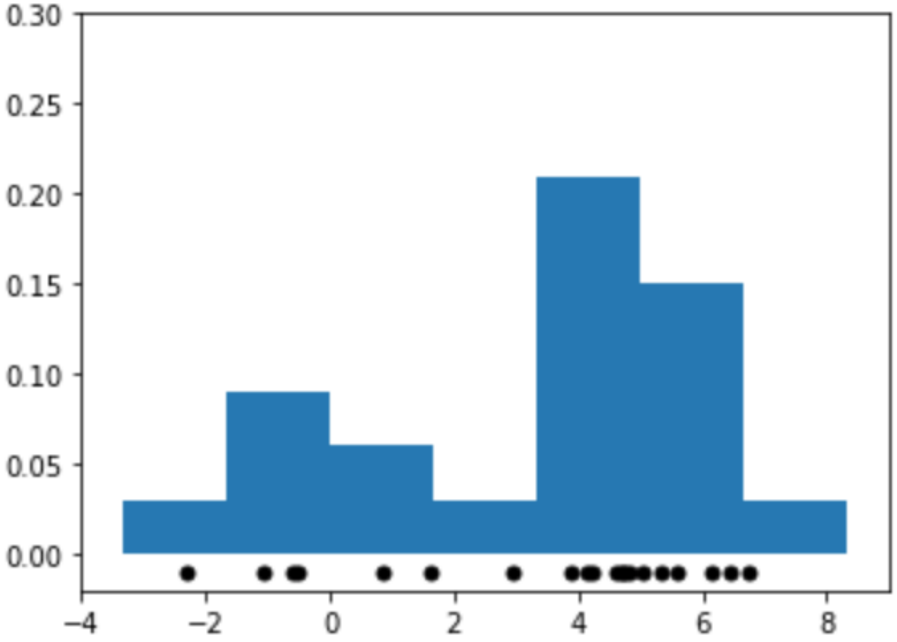
- shifting the bin mean by 0.75

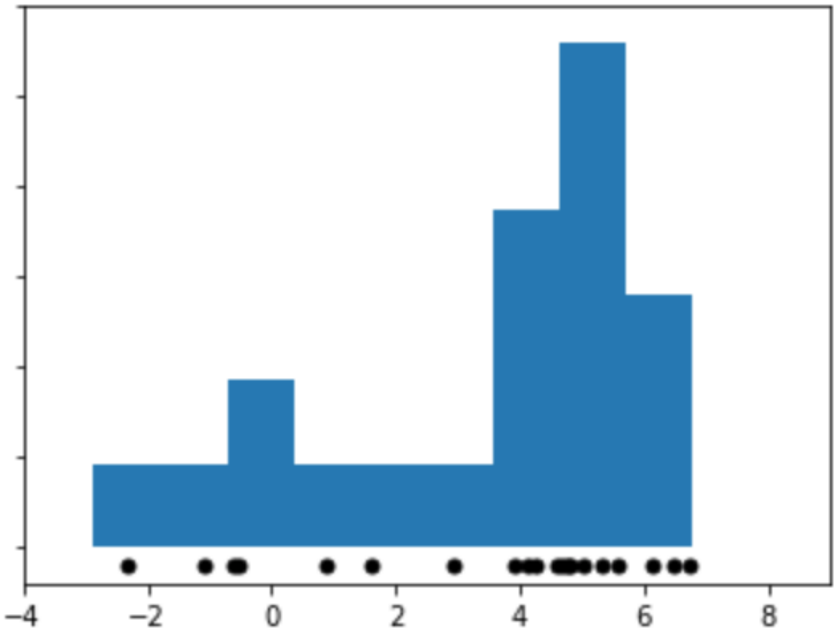




Example: decreased bin width

- decreasing the bin width to 1





Limitations of Histograms

- The histogram is not "smooth".
  - Expect elements in the same local region of the data distribution to have somewhat similar probabilities (no sudden change).
- The shape of the histogram depends on the bin positions and bin width.
- The number of grid cells increases exponentially with dimension d.

Solution: Kernel Density Estimation

- Instead of stacking bins that represent each data point
- We stack distributions
- Example: Gaussian Kernel

